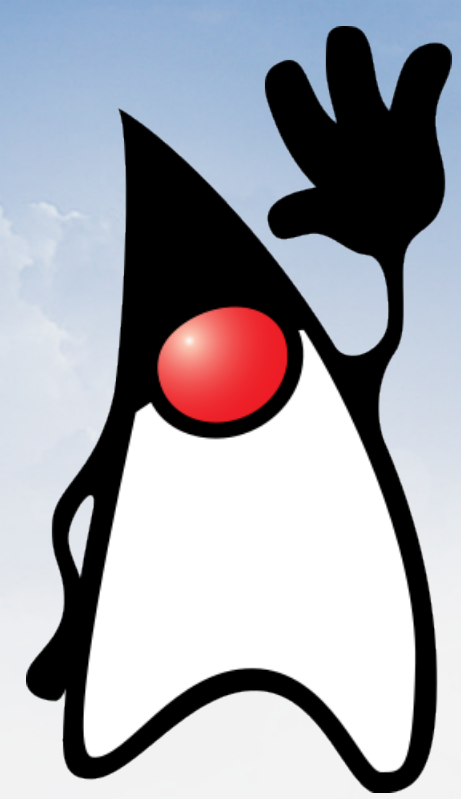




Exceptions

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

What is the difference between $1/0$ and $0/0$?

Considering $x \neq 0$

$$\text{if } \frac{x}{0} = n$$

$$0 \times n = x$$

$$0 = x$$

We have contradiction

$n = \text{undefined}$

$$\text{if } \frac{0}{0} = n$$

$$0 \times n = 0$$

n can be anything

$$n = \infty$$

Exceptions

- Java and most of the other programming languages allow developers to handle unexpected/faulty cases.
- What if the file you want to read does not exist? Readable?
- What if the network packet did not come on time?
- What if the String to be parsed as int, doesn't have number inside?
- Do you prefer to program halt OR continue to run inappropriately OR will you prefer a chance?

Even more if you believe you need to handle some unexpected cases in your code, would you want a mechanism?

Exceptions

- If you are using a method specifying throws XException
 - You are obliged to wrap that method with try and catch block
 - You can also write finally block which is optional.

FileReader

```
public FileReader(File file)  
    throws FileNotFoundException
```

Creates a new `FileReader`, given the `File` to read from.

Parameters:

`file` - the `File` to read from

Throws:

`FileNotFoundException` - if the file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading.

**FileReader constructor is
throwing exception**

```
package javaEx;  
import java.io.*;
```

```
public class TesterCode {  
    public static void main(String[] args)  
    {  
        BufferedReader br;  
        try {  
            FileReader fr=new FileReader("data.txt");  
            br=new BufferedReader(fr);  
            System.out.println("Everything is ok");  
        }catch(FileNotFoundException e) {  
            System.out.println("FILENOTFOUND");  
            e.printStackTrace();  
        }  
        System.out.println("ENDOFBLOCK");  
  
    }  
}  
}
```

Exception handling

- Once an Exception occurs, remaining code is not executed.
- The exception will be handled at the first appropriate case
- The code jumps to after catch!

FILENOTFOUND

```
java.io.FileNotFoundException: data.txt  
(The system cannot find the file  
specified)at
```

```
...
```

```
java.base/java.io.FileReader.<init>(FileReader.java:60)
```

ENDOFBLOCK

```
at javaEx.TesterCode.  
main(TesterCode.java:70)
```

Multiple Catches

- They allow us to handle each specific Exception
 - do necessary action accordingly
 - Log it, or call a different function afterwards etc..
- Ethically and also logically, handling general *Exception* case should be at the last one
 - What happens if it is the first one?
- AS A RULE OF THUMB, do not clean the Catch block
 - Make sure you write some code in the catch block.
 - Do not force the JVM to be silent
 - Is there an exception for this rule?

Multiple Catches

OVERVIEW MODULE PACKAGE **CLASS** USE TREE DEPREC

ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD

Module java.base

Package java.io

Class FileNotFoundException

java.lang.Object

java.lang.Throwable

java.lang.Exception

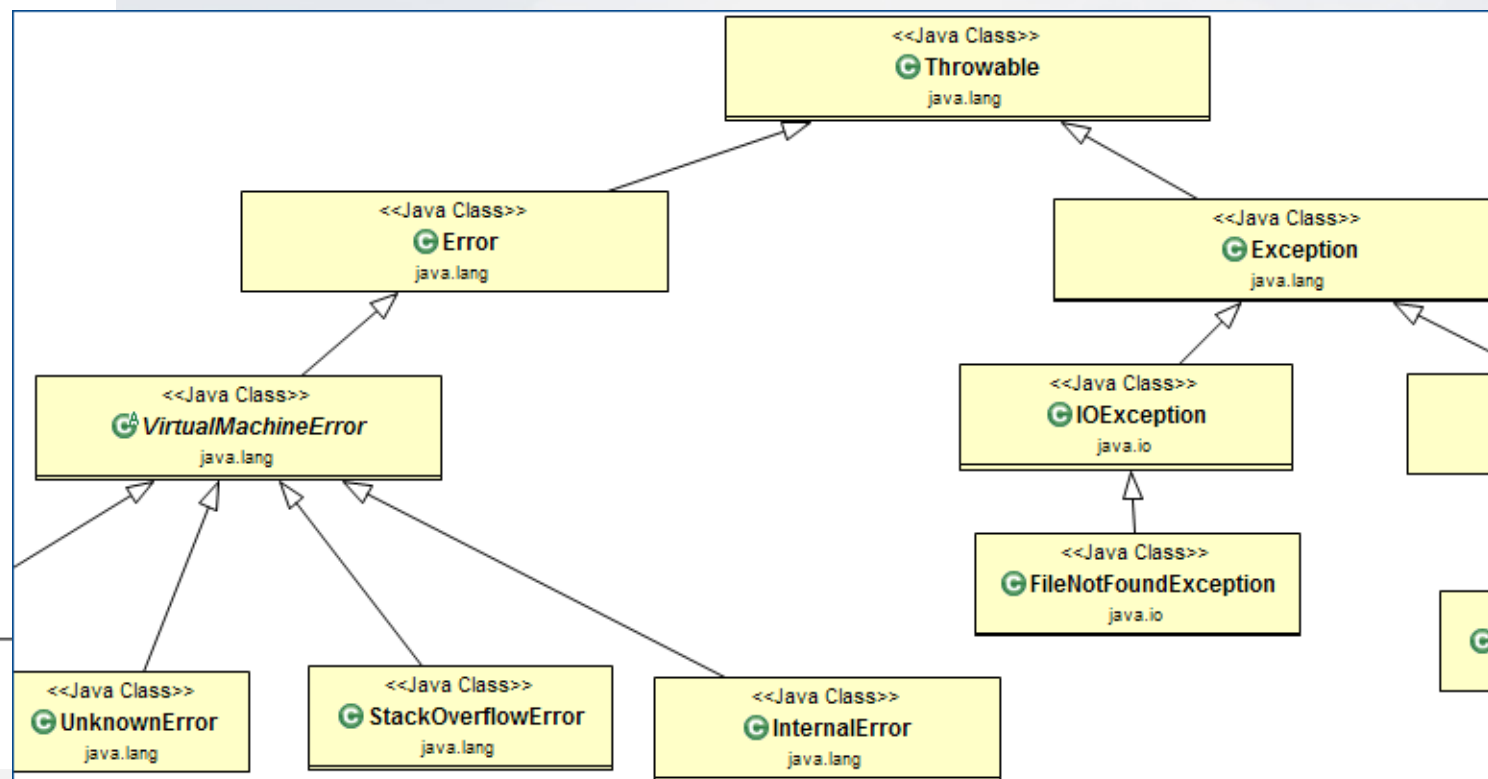
java.io.IOException

java.io.FileNotFoundException

All Implemented Interfaces:

Serializable

```
public class FileNotFoundException  
extends IOException
```



multiple catch statements

```
package javaEx;
import java.io.*;
public class TesterCode {
    public static void main(String[] args)
    {
        try {
            int result = 0/0;
        }catch(ArithmeticException e) {
            System.out.println("Arithmetic");
            e.printStackTrace();
        }catch(Exception e) {
            System.out.println("Exception");
            e.printStackTrace();
        }
        System.out.println("ENDOFBLOCK");
    }
} //end of main
} //end of class
```

ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD | DETAIL

Module java.base

Package java.lang

Class ArithmeticException

java.lang.Object

java.lang.Throwable

java.lang.Exception

java.lang.RuntimeException

java.lang.ArithmeticException

Compiler give a warning: IO
Exception may not be
reachable. It will get
COMPILED FINE.

multiple catch statements

```
package javaEx;
import java.io.*;
public class TesterCode {
    public static void main(String[] args)
    {
        BufferedReader br;
        try {
            FileReader fr=new FileReader("data.txt");
            br=new BufferedReader(fr);
            System.out.println("Everything is ok");
        }catch(FileNotFoundException e) {
            System.out.println("FILENOTFOUND");
            e.printStackTrace();
        }catch(IOException e) {
            System.out.println("IOEXCEPTION");
            e.printStackTrace();
        }catch(Exception e) {
            System.out.println("EXCEPTION");
            e.printStackTrace();
        }
        System.out.println("ENDOFBLOCK");
    }
}
//end of main
//end of class
```

Compiler give a warning: IO
Exception may not be
reachable

multiple catch statements

```
package javaEx;
import java.io.*;
public class TesterCode {
    public static void main(String[] args)
    {
        BufferedReader br;
        try {
            FileReader fr=new FileReader("data.txt");
            br=new BufferedReader(fr);
            System.out.println("Everything is ok");
        }catch(Exception e) {
            System.out.println("EXCEPTION");
            e.printStackTrace();
        }catch(IOException e) {
            System.out.println("IOEXCEPTION");
            e.printStackTrace();
        }catch(FileNotFoundException e) {
            System.out.println("FILENOTFOUND");
            e.printStackTrace();
        }
        System.out.println("ENDOFBLOCK");
    }
}
//end of main
//end of class
```

Compiler give an ERROR.
Unreachable code.

multiple catch statements - Wrong order

```
package javaEx;
import java.io.*;
public class TesterCode {
    public static void main(String[] args)
    {
        try {
            int result = 0/0;
        } catch (Exception e) {
            System.out.println("Exception");
            e.printStackTrace();
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic");
            e.printStackTrace();
        }
        System.out.println("ENDOFBLOCK");
    } //end of main
} //end of class
```

ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD | DETAIL

Module java.base

Package java.lang

Class ArithmeticException

java.lang.Object

java.lang.Throwable

java.lang.Exception

java.lang.RuntimeException

java.lang.ArithmeticException

Compiler give an ERROR.
Unreachable code.

Java's some defaults

Some cases can throw exception but Java allows those not to be wrapped around try-catch. That is called unchecked exception.

IOExceptions are checked. Developer must handle that.

It is left to developer to write try-catch block.

Unchecked Exception giving code examples:

```
int num = Integer.parseInt("JAVA");  
// java.lang.NumberFormatException exception is unchecked
```

```
int num = 1/0;  
//ArithmeticException is unchecked
```

```
package javaEx;
public class TesterCode {
    public static void main(String[] args)
    {
        int result=0;
        try {
            result= 1/0;
            System.out.println("AFTER RESULT");
        }catch(ArithmeticException e) {
            System.out.println("ARITHMETIC");
            e.printStackTrace();
        }catch(Exception e) {
            System.out.println("EXCEPTION");
            e.printStackTrace();
        }finally {
            System.out.println("FINALLY");
        }
        System.out.println(result+ " IS THE RESULT");

    } //end of main
} //end of class
```

Exception running order

finally block is always optional.
Always being executed no MATTER
WHAT HAPPENS!

Finally block is generally used to
close/clean resources. *scan.close();*

ARITHMETIC

java.lang.ArithmeticException: / by zero

at

javaEx.TesterCode.main(TesterCode.java:49)

FINALLY

0 IS THE RESULT

```
package exTester;
import java.io.*;
public class TesterMain {
    public static void main(String[] args){
        int result=0;
        try {
            result= 1/0;
            System.out.println("AFTER RESULT");
        }catch(ArithmeticException e) {
            System.out.println("ARITHMETIC");
            e.printStackTrace();
            return;
        }catch(Exception e) {
            System.out.println("EXCEPTION");
            e.printStackTrace();
        }finally {
            System.out.println("FINALLY");
        }
        System.out.println(result+ " IS THE RESULT");
    } //end of main
} //end of class
```

Exception running order

finally block is always optional.
Always being executed no MATTER
WHAT HAPPENS!

```
ARITHMETIC
FINALLY
java.lang.ArithmeticException: /
by zero
      at
queue4.TesterMain.main(TesterMain
.java:10)
```

This line is not executed.

Why don't we handle everything with ifs?

- Exceptions can be propagated up the call stack. We would be able to see callstack and error message if we use Exceptions.
- Exceptions allow for more flexible error handling strategies, such as retrying an operation, logging errors, or providing alternative behavior. With if statements alone, handling all possible error conditions can become complex and error-prone.
- Ifs can kill the readability of the main logic of the program. We can handle exceptions in exceptions context.

Writing our own Exception

- We may want to create an exception for OutOfFunds or Infinite number
- We will need to manually *throw* new OurException() so that the caller method will put our method in try catch block
- Anything after throw statement is unreachable. Program will leave the method.
- For our method we will need to state that it may throw an exception. We state it by writing *throws*.

Custom Exception

```
package javaEx;
public class SpecialDiv {
    // takes a string as 4:5 and checks the string
    public static double specialDivide(String numbersGiven) throws InfiniteException {
        String[] num=numbersGiven.split(":");
        int returned=0;
        if( num[0] == 0 && num[1] == 0 ) {
            throw new InfiniteException();
        }else {
            returned = num.getFirst() / num.getSecond();
        }
        return returned;
    }
} //end of class
```

```
package javaEx; // Our Custom Exception
public class InfiniteException extends Exception {
    public InfiniteException() {
        super("Infinite Number Exception");
    }
}
```

```
package javaEx;
public class TesterCode {
    public static void main(String[] args)
    {
        ArrayList<String > arr = new ArrayList<>();
        arr.add( "1:0" );
        arr.add( "0:0" );
        for(String inputcase:arr) {
            double result=0;
            try {
                result= SpecialDiv.specialDivide(inputcase);
            }catch(ArithmeticException e) {
                System.out.println("ARITHMETIC");
                e.printStackTrace();
            }catch(Exception e) {
                System.out.println("\nEXCEPTION");
                e.printStackTrace();
            }
            System.out.println("input params " +inputcase+" result is "+result);
        }
    } //end of main
} //end of class
```

We will get Arithmetic
Exception first, then we will
get InfiniteException

Output

ARITHMETIC

```
java.lang.ArithmeticException: / by zero  
at javaEx. SpecialDiv. specialDivide(SpecialDiv.java:9)  
at javaEx. TesterCode. fun1(TesterCode.java:18)  
at javaEx. TesterCode. main(TesterCode.java:71)  
input params (1,0) result is 0.0
```

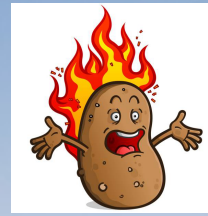
EXCEPTION

```
javaEx.InfiniteException: Infinite Number Exception  
at javaEx. SpecialDiv. specialDivide(SpecialDiv.java:7)  
at javaEx. TesterCode. fun1(TesterCode.java:18)  
at javaEx. TesterCode. main(TesterCode.java:71)  
input params (0,0) result is 0.0
```

```
package javaEx;
public class TesterCode {

    public static void main(String[] args) {
        try {
            thrower2();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    public static void thrower1() throws Exception {
        throw new Exception("Something Happened");
    }
    public static void thrower2() throws Exception {
        thrower1();
        // running away try catch responsibility by
        // writing throws Exception
    }
} //end of class
```

What if the main method also throws Exception?



Hot Potato

- Exception handling is done by hot-potato method.
- If a method throws an Exception it doesn't need to use try catch but the caller need to!
- If the caller also throws Exception then the caller of that will need to handle.

```
java.lang.Exception: Something Happened
at javaEx. TesterCode.
thrower1(TesterCode.java:79)
at javaEx. TesterCode.
thrower2(TesterCode.java:83)
at javaEx. TesterCode.
main(TesterCode.java:72)
```